

1. JVM Memory Architecture

The JVM divides memory into distinct runtime data areas, each serving a specific purpose. Understanding these regions is essential for diagnosing memory issues and writing efficient Java code.

Memory Area	Scope	Stores	GC?
Heap	Shared / JVM-wide	Objects, instance variables, arrays	Yes
Stack (per-thread)	Per thread	Local vars, method frames, partial results	No (LIFO)
Method Area / Metaspace	Shared	Class metadata, static fields, constants	Rarely
PC Register	Per thread	Address of current executing instruction	No
Native Method Stack	Per thread	Native (C/C++) method calls	No

2. Heap Structure (Generational GC)

Java's heap is divided into generations based on the observation that most objects die young (Weak Generational Hypothesis).

Generation	Region	Purpose	GC Type
Young Gen	Eden	New object allocation	Minor GC
Young Gen	Survivor S0/S1	Objects surviving Eden GC	Minor GC
Old Gen	Tenured	Long-lived objects (after N minor GCs)	Major/Full GC
Non-heap	Metaspace (JDK8+)	Class metadata — grows dynamically	Partial
Non-heap	Code Cache	JIT-compiled native code	N/A

✓ Object promotion: Eden → Survivor (after 1st GC) → Tenured (after threshold, default 15 cycles).

3. Stack vs Heap — Quick Comparison

Feature	Stack	Heap
Allocation	Compile-time (LIFO)	Run-time (dynamic)
Speed	Very fast	Slower (GC overhead)
Size	Small (~512 KB–1 MB)	Large (configurable)
Thread safety	Thread-local (safe)	Shared (needs sync)
Stores	Primitives, references	Objects, arrays
Lifetime	Method scope	Until GC collects
Error	StackOverflowError	OutOfMemoryError

4. Garbage Collection (GC)

GC automatically reclaims memory occupied by objects no longer reachable from any GC root. Java uses Tracing GC — it identifies live objects starting from roots and collects the rest.

GC Roots

- Local variables on active thread stacks
- Static variables in loaded classes
- JNI references (native code)
- Active Java threads themselves

GC Phases

Phase	What Happens
Mark	Traverse object graph from GC roots; mark all reachable objects
Sweep	Reclaim memory of unreachable objects
Compact	(Optional) Move live objects together to reduce fragmentation
Promote	Surviving objects moved to older generation after threshold

5. JVM Garbage Collectors

Collector	Flag	Best For	Pause
Serial GC	-XX:+UseSerialGC	Single-threaded / small heaps	High
Parallel GC	-XX:+UseParallelGC	Throughput-focused apps	Medium
CMS (deprecated)	-XX:+UseConcMarkSweepGC	Low-latency (old gen)	Low
G1 GC (default)	-XX:+UseG1GC	Balanced latency+throughput	Low-Medium
ZGC	-XX:+UseZGC	Ultra-low latency, large heaps	Sub-ms
Shenandoah	-XX:+UseShenandoahGC	Low pause, concurrent compaction	Sub-ms

✓ G1 is the default from JDK 9+. ZGC / Shenandoah for JDK 15+ production workloads.

6. Key JVM Memory Flags

Flag	Purpose	Example
-Xms	Initial heap size	-Xms256m
-Xmx	Maximum heap size	-Xmx2g
-Xss	Thread stack size	-Xss512k
-XX:MetaspaceSize	Initial Metaspace size	-XX:MetaspaceSize=128m
-XX:MaxMetaspaceSize	Max Metaspace	-XX:MaxMetaspaceSize=256m
-XX:NewRatio	Old/Young gen ratio	-XX:NewRatio=3
-XX:SurvivorRatio	Eden/Survivor ratio	-XX:SurvivorRatio=8
-XX:MaxTenuringThreshold	Promotions before tenured	-XX:MaxTenuringThreshold=15
-XX:+PrintGCDetails	Print GC logs	Diagnostic
-XX:+HeapDumpOnOutOfMemoryError	Dump heap on OOM	Diagnostic

7. Object Lifecycle in Java

Stage	Description
1. Creation	new keyword allocates in Eden; constructor runs

2. In Use	Referenced by at least one GC root; object is live
3. Invisible	No references reachable but finalizer not yet run
4. Unreachable	No references from GC roots; eligible for collection
5. Collected	Finalizer (if any) run; memory reclaimed by GC
6. Finalized	finalize() completed (deprecated in Java 9+, removed 18)

■ Avoid finalize() — use try-with-resources or Cleaner API instead.

8. Reference Types (java.lang.ref)

Type	Class	GC Behaviour	Use Case
Strong	(default)	Never collected while reachable	Normal usage
Soft	SoftReference<T>	Collected only when OOM imminent	Memory-sensitive caches
Weak	WeakReference<T>	Collected on next GC cycle	Canonical maps, listeners
Phantom	PhantomReference<T>	After finalization; for cleanup	Resource cleanup (Cleaner)

```
WeakReference ref = new WeakReference<>(new MyObj());
```

```
SoftReference cache = new SoftReference<>(new byte[1024]);
```

9. Common Memory Problems & Fixes

Problem	Cause	Fix
Memory Leak	Objects retained unintentionally (static collections, listeners)	Use cleaners; remove listeners; scope variables tightly
OutOfMemoryError: Java heap space	Heap too small or genuine leak	Increase -Xmx; analyse heap dump
OutOfMemoryError: Metaspace	Too many classes / classloaders	Set -XX:MaxMetaspaceSize; fix classloader leaks
StackOverflowError	Infinite or very deep recursion	Refactor to iteration; increase -Xss
Excessive GC pauses	Too-small heap or wrong GC	Tune heap ratios; switch to G1/ZGC
Premature Promotion	Short-lived objects reaching OldGen	Increase survivor space / Eden size

10. Memory-Efficient Code Patterns

Use try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    return br.readLine();
} // auto-closes; no finally block needed
```

StringBuilder over String concatenation in loops

```
StringBuilder sb = new StringBuilder();
for (String s : list) sb.append(s); // O(n) not O(n^2)
String result = sb.toString();
```

Object Pooling (e.g., DB connections)

```
// Use HikariCP / c3p0 – avoid creating connections every request
DataSource ds = new HikariDataSource(config);
```

Avoid static collections holding live references

```
// BAD: static List cache = new ArrayList<>();
// GOOD: static Map<> cache = new WeakHashMap<>();
```

11. Profiling & Diagnostic Tools

Tool	Type	Use
jps	CLI	List running JVM processes
jmap	CLI	Heap dump, class histogram
jstat	CLI	GC stats, heap usage live
jconsole	GUI	Real-time memory / thread monitoring
VisualVM	GUI	Heap dump analysis, CPU/mem profiling
Eclipse MAT	GUI	Deep heap dump analysis, leak suspects
Java Flight Recorder	GUI/CLI	Low-overhead production profiling (JDK 11+)
async-profiler	CLI/GUI	CPU + allocation flame graphs

```
# Heap dump
jmap -dump:live,format=b,file=heap.hprof
# GC stats every 1s
jstat -gcutil 1000
```

12. Best Practices

- Set -Xms == -Xmx in production to prevent heap resizing pauses.
- Use G1GC (default JDK 9+); switch to ZGC for latency-sensitive services.
- Always close I/O resources with try-with-resources.
- Nullify references in long-lived data structures once no longer needed.
- Prefer primitive types (int, long) over boxed (Integer, Long) in hot paths.
- Use WeakHashMap for caches keyed by objects you don't own.
- Avoid finalizers — use java.lang.ref.Cleaner (Java 9+).
- Measure before optimising — profile with JFR or VisualVM first.
- Keep object graphs shallow; deep object trees increase GC scan time.
- Use -XX:+HeapDumpOnOutOfMemoryError in production for post-mortem analysis.

13. Cheat Sheet — At a Glance

Topic	Key Point
Heap	Objects live here; divided into Young / Old gen
Stack	Method frames + local vars; one per thread; fast; LIFO
Metaspace	Class metadata; unlimited by default (JDK 8+) — cap it!
Minor GC	Cleans Young gen; fast; frequent
Major/Full GC	Cleans Old gen (+ Young); slow; avoid frequent occurrence
G1 GC	Region-based; predictable pauses; default JDK 9+
ZGC / Shenandoah	Concurrent compaction; sub-millisecond pauses; JDK 15+
Strong ref	Default; prevents GC

Weak ref	Collected on next GC; use for caches
Soft ref	Collected on OOM; ideal for memory-sensitive caches
OOM: heap space	Increase -Xmx or fix leak
OOM: Metaspace	Set -XX:MaxMetaspaceSize
StackOverflow	Deep recursion → refactor or increase -Xss
-Xms / -Xmx	Initial / max heap — set equal in prod
Heap dump	jmap or -XX:+HeapDumpOnOutOfMemoryError